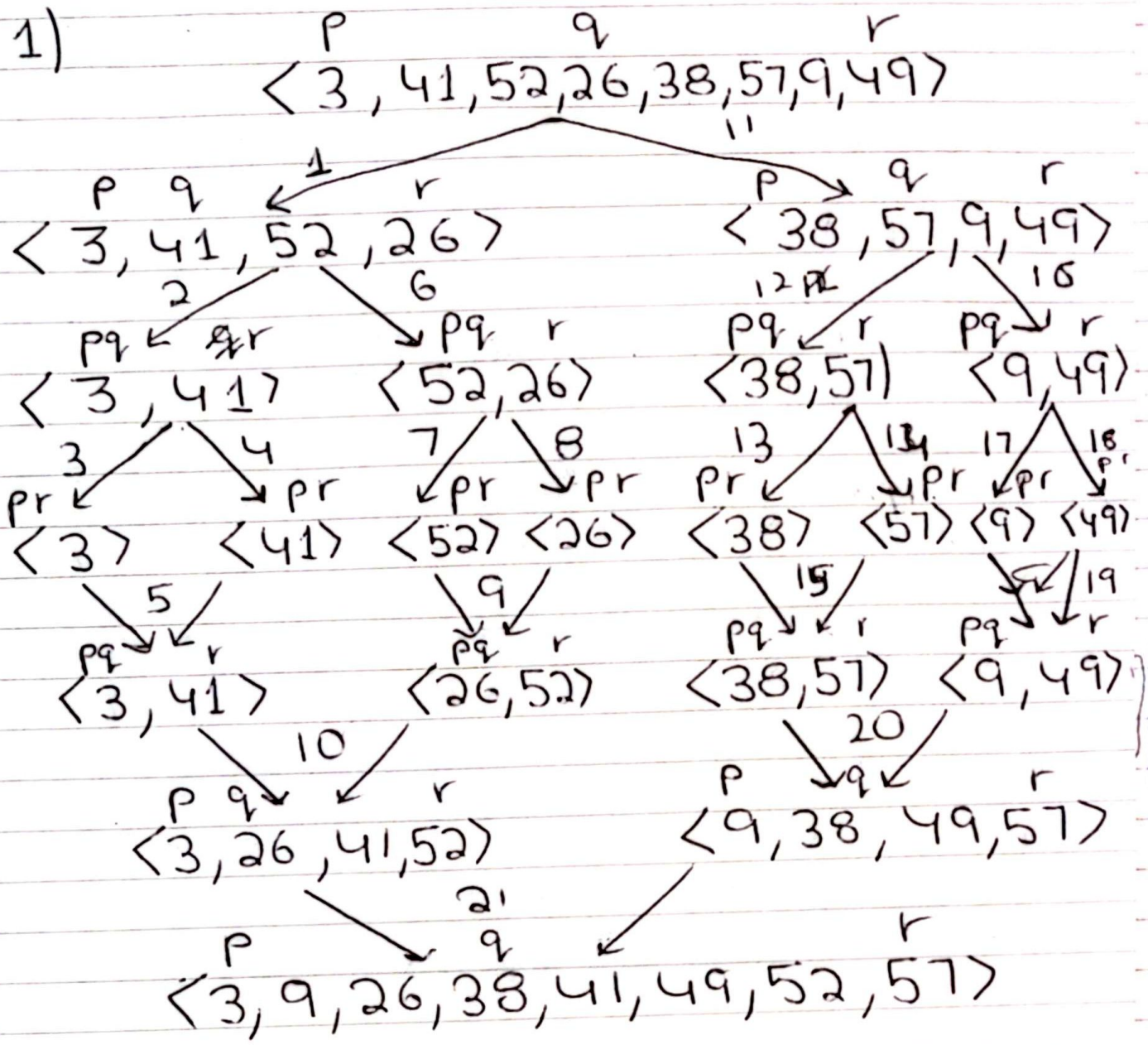
Exercises 2.3

1)



2) ~~If the initial call MERGE-SORT(A, 1, n) has $n \geq 1$, then writing~~

Note:- Question should read '$p == r$' not '$p \neq r$'.

a) ~~We rewrite merge sort with such that "if $p \neq r$, recurse, else return".~~

~~Then~~ If we call merge sort with $(A, 1, n)$ where $n \geq 1$, we have two possible situations:-

Case 1:- $n==1$

Then we do not recurse and the merge sort function simply 'returns'.

Case 2:- $n>1$

Then ~~the~~ $p \neq r$ and so the main function body runs; q is calculated as $\lfloor (1+n)/2 \rfloor$ which always gives $q \geq 1$ when $n>1$, and then we have a recursive call 'MERGE-SORT (A, 1, q)' where $q \geq 1$ (and so $p = 1 \leq q = r$). Once this recursive call is complete, there is a second recursive call 'MERGE-SORT (A, q+1, r)' ~~and~~ (where ~~since q+~~ $p = q+1 \leq r$). Hence there are no recursive calls ~~with~~ MERGE-SORT (A, p, r) such that $p > r$.

3) ~~P~~The loop invariant for the while loop of lines 12-18 is:-

"Before each iteration, the subarray $A[p : k-1]$ contains the elements of $L[0 : i-1]$ and $R[0 : j-1]$ in sorted order"

Once the while loop terminates, either $i = n_L$, $j = n_L$ or both. ~~If~~

~~$i = n_L$, then we still have $j < n_R$ and so t and the subarray $A[0:k-1]$ containing elements~~

If $i = n_L$, then the subarray $A[1:k-1]$ has all of the elements of L and perhaps some of the smallest elements of R in sorted order, and so we just add the remaining elements of R via the third while loop and the algorithm is correct. If $j = n_R$, then the subarray $A[1:k-1]$ has all of the elements of R and perhaps some of the smallest elements of L in sorted order, so we just add the remaing elements of L via the second while loop and so the algorithm is correct. If both $i = n_L$ and $j = n_L$, then the subarray $A[1:k-1]$ has all the elements of L and R in ~~corr~~ sorted order and so the algorithm is correct.

4) Base case:- Prove $T(2) = 2\log_2(2)$

$T(2)$ is given to be 2, and $2\log_2(2) = 2$, hence the base case is true.

$= n\log_2(n)$

Inductive step:- Proof if $T(n)$ then $T(n \times 2)$ ~~where $n \geq 2$~~ $= 2n\log_2(2n)$

Suppose $T(n) = n\log_2(n)$ for some $n \geq 2$ such that n is an exact power of 2. (Inductive hypotheses)

Then by definition, $n = \cancel{k}\, 2^k$ for some ~~positinteger~~ positive integer k, and the next power of 2 is hence $2^{k+1} = 2k \times 2 = n \times 2$.

By the recurrence, $T(2n) = 2T\left(\frac{\cancel{2}n}{\cancel{2}}\right) + 2n$

Applying the IH, we get:-

$$T(2n) = 2\cancel{T} n\log_2(n) + 2n$$

~~Note~~ Note that $2n\log_2 \cancel{(n)} + (2n)$

$$= 2n(\log_2(2) + \log_2(n))$$
$$= 2n(1 + \log_2(n))$$
$$= 2n + 2n\log_2(n)$$

Hence $T(2n) = 2n\log_2 \cancel{(n) + 2n}\ (2n)$

5)

5)

```
RECURSIVE-INSERTION-SORT(A,n)
if n<=1
    return
RECURSIVE-INSERTION-SORT(A,n-1)
key = A[n]
j=n-1
while j>0 and A[j] > key
        A[j+1] = A[j]
        j= j-1
A[j+1] = key
```

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ T(n-1) + \Theta(n) & \text{if } n>1 \end{cases}$$

6)

```
BINARY-SEARCH (A,n,v)
   left = 1
   right = n
   while left <= right
        mid = ⌊(left + right)/2⌋
        if v == A[mid]
            return mid
        else-if v < A[mid]
            right = mid - 1
        else-if v > A[mid]
            left = mid + 1
   return NIL
```

The worst case running time occurs when we have to do a maximum number of 'midpoint check', which is the maximum number of times we can divide ~~the~~ the array in half, which is given ~~directly~~ by $\log_2 n$, and so the worst case running time is $\Theta(\log n)$

7) In the worst case, the input array is in reverse sorted order, so binary search on the subarray will always return index 1 as the index where the new element is to be inserted, and that will always require shifting the rest ~~of~~ of the subarray ~~to~~ one index to the right thus taking time $\Theta(n)$ i.e. no improvement.

8)

```
SUM-SEARCH (S, n, x)
// Set 'S' is represented as an Array
MERGE-SORT (S, 1, n)
Left = 1
right = n
while left < right
        sum = S[left] + S[right]
        if sum == x
            return TRUE
        else-if sum < x
            left = left + 1
```

```
else-if sum > x
        right = right - 1
return FALSE
```